

Основи програмування графічних інтерфейсів

До цього часу користувач взаємодіяв з програмами через консоль, тобто передавав програмі вхідні дані та отримував результат через текстове вікно командної оболонки. Реальні програмні продукти зазвичай взаємодіють з користувачами через віконні графічні інтерфейси користувача (GUI), маніпулюючи інформацією, використовуючи меню, кнопки, піктограми, перемикачі та інші елементи вікна (віджети).

На відміну від консольних, графічні інтерфейси, крім візуалізації, дають змогу вводити вхідну інформацію у довільному порядку і виконувати певні операції після натискання відповідних кнопок (комбінацій клавіш) чи вибору відповідних команд з меню. Крім того, вони позбавляють необхідності перезапускати програму і повторно вводити вхідні дані, коли потрібно отримати результат на різних наборах даних.

Для розробки графічних інтерфейсів Python надає стандартну бібліотеку `tkinter`, яка містить готові класи для вікон та віджетів. Продемонструємо принципи програмування деяких елементів графічного інтерфейсу з використанням цієї бібліотеки на прикладі програми для обчислення площі круга та довжини кола вказаного радіуса, вікно якої зображене на рис. 1.

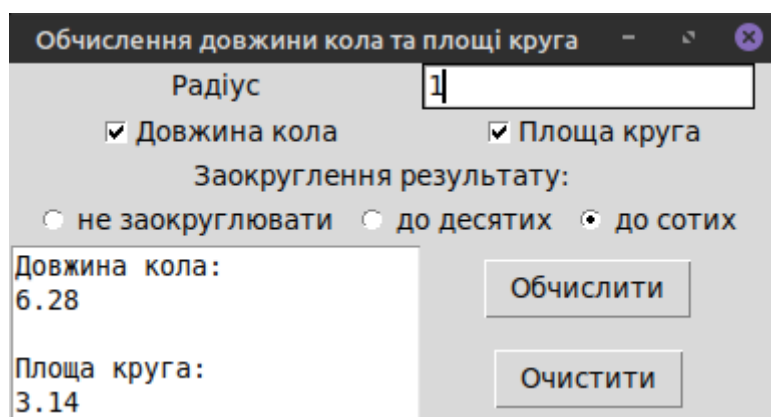


Рис. 1 - Приклад графічного інтерфейсу

Кореневе вікно

Для створення кореневого вікна, у якому розміщуватимуться віджети, використовується клас Tk. Цей клас має методи як для задання певних атрибутів вікна (наприклад, заголовку, початкових розмірів, вигляду керуючих кнопок), так і метод `mainloop()`, призначенням якого є постійне відображення вікна на екрані та його зміна за умови настання певних подій (введення інформації, натискання кнопки, наведення вказівника миші тощо). Зовнішній вигляд вікна відповідає вікнам операційної системи, у якій виконується програма.

Наступний код створює порожнє вікно програми для обчислення довжини кола та площі круга, яке зображене на рис. 2:

```
from tkinter import *

root = Tk()
root.title('Обчислення довжини кола та площі круга')
root.mainloop()
```

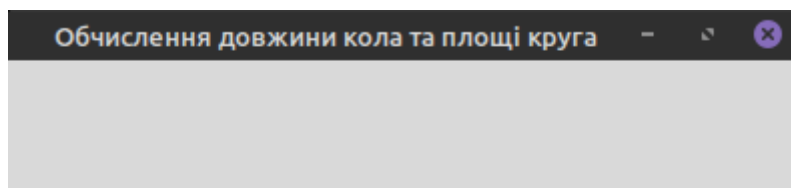


Рис. 2 - Кореневе вікно

Будь-який віджет, який розміщуватиметься у вікні, незалежно від своєї форми, займає прямокутну область. Тому розташування (компонування) віджетів у вікні зводиться до поділу вікна на прямокутники, по одному для кожного віджету.

Бібліотека `tkinter` пропонує два методи компонентування віджетів — `grid()` і `pack()`. Перший з них полягає в умовному поділі вікна на сітку з пронумерованими (починаючи з 0) рядками і стовпчиками, другий — у поступовому впорядкованому заповненні вільного простору вікна віджетами шляхом вказування їх місця прикріплення.

Розглянемо метод компонентування `grid()`, тобто місце розташування віджета у вікні задається номерами рядка і стовпчика.

Напис (віджет Label)

Напис у вікні графічного інтерфейсу створюється за допомогою класу `Label`. Першим аргументом вказується, у якому контейнері має бути розміщений напис (у нашому випадку це вікно `root`), а іменованим аргументом `text` передається власне текст напису.

Для розміщення напису в конкретному місці вікна аргументами `row` і `column` методу `grid()` передаються номери рядка і стовпця:

```
radiusLabel = Label(root, text='Радіус')
radiusLabel.grid(row=0, column=0)
```

У результаті матимемо вікно з написом, зображене на рис. 3.

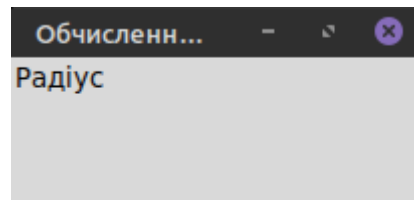


Рис. 3 - Вікно з написом

Якщо наперед відомо, що напис ніколи не змінюватиметься впродовж виконання програми, то допускається запис коду в один рядок:

```
Label(root, text='Радіус').grid(row=0, column=0)
```

Умова незмінюваності віджета впродовж виконання програми накладається тому, що після такого однорядкового запису коду доступ до об'єкта, а отже і до його атрибутів буде втрачено, навіть якщо запис виглядатиме так:

```
radiusLabel = Label(root, text='Радіус').grid(row=0, column=0)
```

Поле для введення (віджет Entry)

У простих графічних інтерфейсах введення текстової інформації зазвичай здійснюється у однорядкове поле для введення, яке створюється за допомогою класу `Entry` (див. рис. 4):

```
radiusEntry = Entry(root)
radiusEntry.grid(row=0, column=1)
```

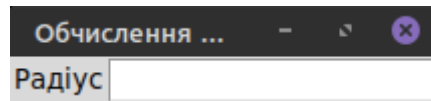


Рис. 4 - Вікно з доданим полем для введення текстової інформації

При цьому у змінній `radiusEntry` не міститиметься введене значення радіусу. Для його отримання потрібно скористатися методом `get()`:

```
radiusEntry.get()
```

Прапорець (віджет `CheckButton`)

Прапорці дають змогу реалізувати інтерфейс для вибору кількох варіантів. Так, у нашій програмі вони дають змогу обчислювати довжину кола, площу круга або обидві величини. Кожен з прапорців є окремим віджетом `Checkbutton`, який асоціює свій стан з певним значенням прив'язаної до нього змінної, ім'я якої передається іменованим аргументом `var`, а значення — іменованим аргументом `value`. Іменований аргумент `text` визначає мітку (напис біля прапорця).

Додати два прапорці опцій з мітками "Довжина кола" і "Площа круга" (див. рис. 5) можна, наприклад, так:

```
calcL = IntVar()
calcS = IntVar()
Checkbutton(root, text='Довжина кола',
            var=calcL).grid(row=1, column=0)
Checkbutton(root, text='Площа круга',
            var=calcS).grid(row=1, column=1)
```

Кожен з прапорців асоціюється зі своєю змінною описаного в бібліотеці `tkinter` типу `IntVar`. Якщо прапорець активувати, то всередині квадрата з'явиться галочка, а значення відповідної змінної стане 1. Значення змінної, асоційованої з неактивованим прапорцем, дорівнює 0. Для отримання поточного стану цих значень використовується метод `get()`, наприклад:

```
calcS.get()
```

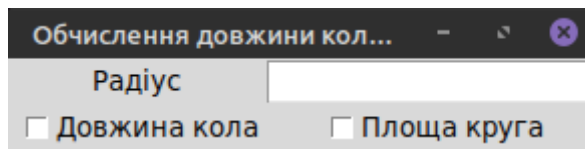


Рис. 5 - Рисунок з доданими прапорцями

Значення аргументів методу `grid()` вказують, що прапорці розміщуються в рядку номер 1 і стовпчиках під номерами 0 та 1 відповідно.

Розміщення віджета у кількох рядках/стовпчиках

Для того, щоб розмістити віджет більше ніж у одному рядку чи стовпчику, методу `grid()` передаються параметри `rowspan` і `columnspan`, які вказують, на скільки рядків чи стовпців відповідно потрібно розтягнути віджет. У нашому графічному інтерфейсі напис "Заокруглення результату:" простягається на два стовпці (див. рис. 1 і рис. 6). Цього можна досягти так:

```
Label(root, text='Заокруглення результату:').grid(row=2,
                                                    column=0,
                                                    columnspan=2)
```

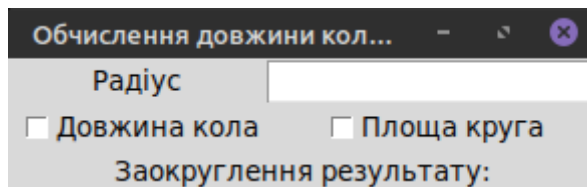


Рис. 6 - Вікно з віджетом-написом завширшки два стовпчики

Контейнер (віджет Frame)

З рис. 1 видно, що способи заокруглення відображені у трьох стовпчиках, а основна сітка кореневого вікна налічує два. Для вирішення цієї невідповідності призначені контейнери (Frame).

Контейнер можна розглядати як звичайне вікно, розташоване всередині іншого вікна (у нашому випадку кореневого). Всередині контейнера можна утворювати нову сітку і компонувати віджети уже в ній, передаючи першим параметром ім'я цього контейнера).

Створимо в наступному рядку сітки кореневого вікна контейнер та розмістимо його на два стовпці (тобто на всю ширину інтерфейсу) вшир.

Оскільки контейнер надалі використовуватиметься, використовувати однорядковий запис коду не можна, щоб не втратити доступ до нього:

```
rounding = Frame(root)
rounding.grid(row=3, column=0, columnspan=2)
```

Перемикач (віджет RadioButton)

Перемикачі дають змогу реалізувати інтерфейс для вибору одного з кількох варіантів. Так, у нашій програмі вони дають змогу або не заокруглювати обчислені значення довжини кола та площі круга, або заокруглювати до десятих, або до сотих.

Кожен з перемикачів у групі є окремим віджетом Radiobutton, але всі вони асоціюються з однією й тією самою змінною, ім'я якої передається передається іменованим аргументом `var`, а значення — іменованим аргументом `value`. Іменований аргумент `text` визначає мітку (напис біля перемикача).

Додати три перемикачі з мітками “не заокруглювати”, “до десятих” і “до сотих” (див. рис. 7) можна, наприклад, так:

```
roundTo = StringVar()
roundTo.set('без заокруглення')

rb1 = Radiobutton(rounding, var=roundTo, value='без заокруглення')
rb1['text'] = 'не заокруглювати'
rb1.grid(row=0, column=0)
rb2 = Radiobutton(rounding, var=roundTo, value='1')
rb2['text'] = 'до десятих'
rb2.grid(row=0, column=1)
rb3 = Radiobutton(rounding, var=roundTo, value='2')
rb3['text'] = 'до сотих'
rb3.grid(row=0, column=2)
```

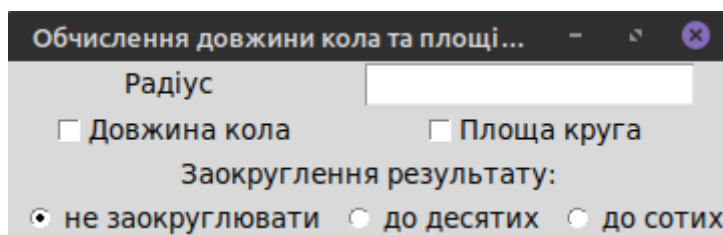


Рис. 7 - Вікно з доданою групою перемикачів

Як бачимо, першим параметром, який передається перемикачеві, є не кореневе вікно, а контейнер, створений у попередньому підрозділі.

Кожен з трьох перемикачів асоціюється зі змінною `roundTo` типу `StringVar` (описаний в бібліотеці `tkinter` тип, аналогічний до типу `str`, але зі своїми методами). Активація перемикача присвоює цій змінній відповідне цьому перемикачу значення `value` (для першого перемикача це 'без заокруглення', для другого та третього — кількість десяткових знаків, які будуть використовуватися під час заокруглення вбудованою функцією `round()`, але у форматі рядків (через тип `StringVar`): '1' та '2' відповідно).

З метою демонстрації іншого способу задання атрибутів віджета, атрибут `text` заданий у окремому рядку.

Методом `set()` у другому рядку коду задається значення перемикача за замовчуванням.

Кнопка (віджет `Button`)

Кнопка (`Button`) — віджет, натискання якого викликає функцію, ім'я якої передається іменованим аргументом `command`. Графічний інтерфейс нашої програми містить кнопки "Обчислити" та "Очистити" (див. рис. 1). Натискання першої викликатиме виконання обчислень і запис результату у відповідне поле (буде створене в наступному підрозділі), а друга — очищення полів з вхідними даними (радіус) та результатом (довжину кола і площу круга) для забезпечення можливості нових обчислень без повторного запуску програми.

Створення кнопок нічим не відрізняється від створення розглянутих вище віджетів (результат зображений на рис. 8):

```
Button(root, text='Обчислити',  
        command=calculate).grid(row=4, column=1)  
Button(root, text='Очистити',  
        command=clear).grid(row=5, column=1)
```

Щоб програма не видавала помилки, потрібно описати асоційовані з цими кнопками функції `calculate()` і `clear()`. Зазвичай вони описуються в іншому файлі (тоді потрібен імпорт) або в поточному файлі одразу після інструкцій імпорту:

```
def calculate():  
    pass  
  
def clear():  
    pass
```

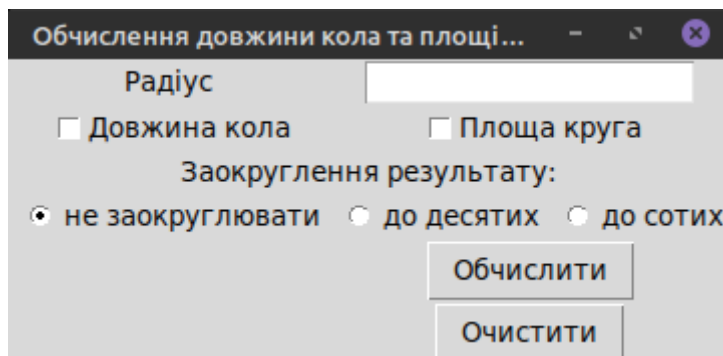


Рис. 8 - Вікно з доданими кнопками

Тіла цих функцій будуть наведені нижче, в підрозділі “Обробка подій”, оскільки в них використовуватиметься наразі не створене поле для результату. Тому на даному етапі просто прописується порожня інструкція `pass`, яка дає змогу коректно запустити програму. Іншими словами, поточний код означає, що при натисканні на кнопки не відбуватиметься нічого, але сам процес натискання дозволений.

Текстове поле (віджет Text)

Текстове поле (віджет Text) у найпростішому випадку використовується для роботи з багаторядковими рядками, тобто значеннями типу `str`, які містять символи кінця рядка `\n`. При цьому індексуються не просто порядкові номери символів, а номери рядків (починаються з 1) та номери стовпців (починаються з 0).

Для простих операцій з текстовим полем (екземпляром класу `Text`) використовують методи `insert()`, `delete()` та `get()`.

Методу `insert()` першим параметром передається позиція, у яку має вставитись рядок, переданий у другому параметрі. Наприклад, код

```
someText = Text()  
someText.insert('1.2', 'Деякий текст')
```

означає, що текст 'Деякий текст' буде вставлений, починаючи з позиції “перший рядок, третій стовпчик”.

Для позначення позицій також використовуються символічні імена. Наприклад, для вилучення всього тексту з текстового поля `someText` методу `delete()` можна передати початкову позицію у формі '<номер рядка>.<номер стовпчика>', а кінцеву у вигляді символічного імені `END`:

```
someText.delete('1.0', END)
```

Метод `get()` використовується для отримання частини тексту між початковою та кінцевою позиціями, наприклад:

```
someText.get('1.2', '2.1')
```

Створимо на основі класу `Text` текстове поле для відображення результатів обчислення (див. рис. 9):

```
result = Text(root, width=25, height = 5)
result.grid(row=4, column=0, rowspan=2)
```

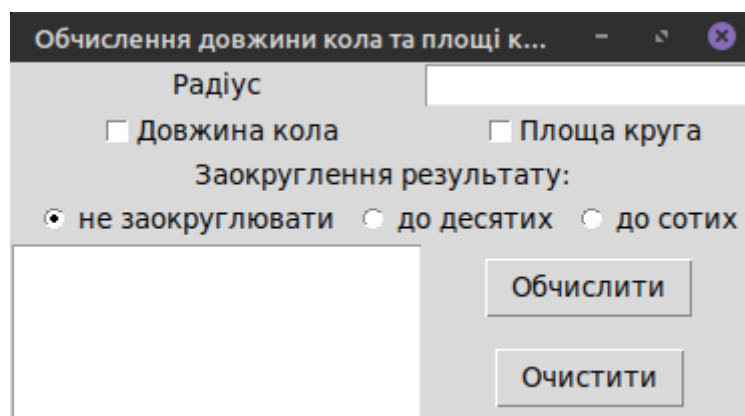


Рис. 9 - Вікно з усіма елементами графічного інтерфейсу

Параметрами `width` і `height` задано точні розміри поля (5 рядків по 25 символів у кожному), оскільки за замовчуванням вони значно більші. Код для запису результату в це поле буде сформований у наступному підрозділі.

Обробка подій

Бібліотека `tkinter` дає змогу обробляти велику кількість подій, прив'язаних до різноманітних змін у графічному інтерфейсі: наведення вказівника миші, клацання певною кнопкою миші на об'єкті, натискання клавіші на клавіатурі, зміна поля, прапорця чи перемикача тощо.

В рамках нашої програми обробка подій стосується виклику функції `clear()` при натисканні кнопки (клацанні лівою кнопкою миші на кнопці) "Очистити" та функції `calculate()` при натисканні кнопки "Обчислити". Вище, під час створення кнопок, ми прив'язали ці функції до відповідних їм кнопок за допомогою іменованого аргумента `command`. Тепер наведемо код тіл цих функцій з коментарями (використання числа "пі" у тілі функції `calculate()` вимагає додаткового імпорту, тобто інструкції `from math import pi`):

```
def calculate():
    # отримуємо текстове значення радіуса з поля для введення та
    # перетворюємо його в дійсне число
    r = float(radiusEntry.get())
    # отримуємо значення асоційованої з перемикачами змінної, щоб
    # знати, яким чином здійснювати заокруглення
    rb = roundTo.get()
    # створюємо порожній рядок для формування тексту результату
    text = ''
    # для зменшення обсягу коду обчислимо заздалегідь як довжину
    # кола, так і площу круга, щоб не використовувати
    # зайвих умовних інструкцій
    l = 2*pi*r          # довжина кола
    s = pi*r**2         # площа круга
    if rb != 'без заокруглення': # якщо потрібно заокруглювати,
        # то заокруглюємо до вказаної кількості десяткових знаків,
        # перетворених в ціле число
        l = round(l, int(rb))
        s = round(s, int(rb)) #
    if calcL.get(): # якщо активований прапорець "Довжина кола",
        # то додаємо текст результату
        text += f'Довжина кола:\n{l}\n\n'
    if calcS.get(): # якщо активований прапорець "Площа круга", то
        text += f'Площа круга:\n{s}' # додаємо текст результату
    # вставляємо результат в текстове поле
    result.insert('1.0', text)

def clear():
    # очищаємо поле для введення радіуса
    radiusEntry.delete(0, END)
    # очищаємо поле з результатом
    result.delete('1.0', END)
```

Важливо! Якщо не використовувати метод `get()`, тобто замість інструкції `if calcL.get():` вживати інструкцію `if calcL:`, то програма не видасть синтаксичної помилки чи помилки виконання, але умова виконуватиметься

завжди, оскільки об'єкт з іменем `calCL` існує. За таких обставин матимемо семантичну помилку, яка призведе до некоректної роботи програми (незалежно від активованих прапорців завжди обчислюватиметься і довжина кола, і площа круга).

Стандартні діалоги

Реальні програми містять стандартні діалоги: вибір файлу, повідомлення про помилки тощо. Усі вони блокують головне вікно програми та повертають керування тільки після закриття вікна діалогу. Бібліотека `tkinter` пропонує набір типових стандартних діалогів. Наприклад, модуль `messagebox` містить функції для реалізації вікон таких діалогів, серед яких: `askyesno()`, `showwarning()`, `showinfo()`. Перша з них створює вікно з текстом та двома кнопками типу "ОК" та "Відміна" (написи на кнопках різняться залежно від операційної системи, під якою працює програма). Дві наступні відображають вікно з текстовим повідомленням і відрізняються лише піктограмами біля тексту. Першим параметром цим функціям передається заголовок вікна, а другим — власне текст повідомлення.

Реалізуємо в нашій програмі обробку винятків, яка в разі винятку відображатиме вікно стандартного діалогу, а саме: якщо в поле для введення радіуса буде введено не число, то після натискання на кнопку "Обчислити" програма виводитиме на екран вікно з відповідним повідомленням-попередженням (див. рис. 10). Для цього імпортуємо функцію `showwarning()` з модуля `messagebox` бібліотеки `tkinter`:

```
from tkinter.messagebox import showwarning
```

Після цього у функції `calculate()` рядок, у якому введене в поле для введення значення радіуса перетворюється в дійсне число переносимо в блок `try`, а у блоці `except` прописуємо відображення діалогового вікна "Некоректні дані" з текстом "Радіус повинен бути числом" та завершуємо виконання функції:

```
try:
    r = float(radiusEntry.get())
except:
    showwarning('Некоректні дані', 'Радіус повинен бути числом')
    return None
```

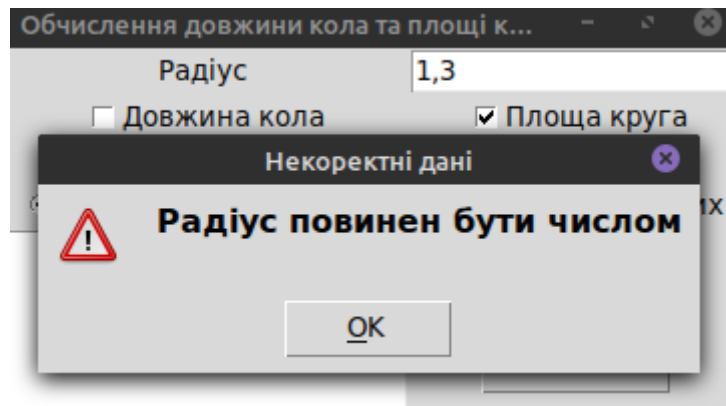


Рис. 10 - Вікно стандартного діалогу

Отже, остаточний код програми для обчислення довжини кола та площі круга за заданим радіусом набуде вигляду (усі коментарі опущені):

```
from tkinter import *
from tkinter.messagebox import showwarning
from math import pi

def calculate():
    try:
        r = float(radiusEntry.get())
    except:
        showwarning('Некоректні дані',
                    'Радіус повинен бути числом')
    return None

    rb = roundTo.get()
    text = ''
    l = 2*pi*r
    s = pi*r**2

    if rb != 'без заокруглення':
        l = round(l, int(rb))
        s = round(s, int(rb))

    if calcL.get():
        text += f'Довжина кола:\n{l}\n\n'

    if calcS.get():
        text += f'Площа круга:\n{s}'

    result.insert('1.0', text)
```

```
def clear():
    radiusEntry.delete(0, END)
    result.delete('1.0', END)

root = Tk()
root.title('Обчислення довжини кола та площі круга')

Label(root, text='Радіус').grid(row=0, column=0)

radiusEntry = Entry(root)
radiusEntry.grid(row=0, column=1)

calcL = IntVar()
calcS = IntVar()

Checkbutton(root, text='Довжина кола',
             var=calcL).grid(row=1, column=0)
Checkbutton(root, text='Площа круга',
             var=calcS).grid(row=1, column=1)

Label(root, text='Заокруглення результату:').grid(row=2, column=0,
                                                    columnspan=2)

rounding = Frame(root)
rounding.grid(row=3, column=0, columnspan=2)

roundTo = StringVar()
rb1 = Radiobutton(rounding, var=roundTo, value='без заокруглення')
rb1['text'] = 'не заокруглювати'
rb1.grid(row=0, column=0)
rb2 = Radiobutton(rounding, var=roundTo, value='1')
rb2['text'] = 'до десятих'
rb2.grid(row=0, column=1)
rb3 = Radiobutton(rounding, var=roundTo, value='2')
rb3['text'] = 'до сотих'
rb3.grid(row=0, column=2)
roundTo.set('без заокруглення')

Button(root, text='Обчислити',
        command=calculate).grid(row=4, column=1)
Button(root, text='Очистити', command=clear).grid(row=5, column=1)

result = Text(root, width=25, height = 5)
result.grid(row=4, column=0, rowspan=2)

root.mainloop()
```